

Unit-4 JavaScript Objects

4.1 Creating Object:

➤ Creating object By object literal

- This is the easiest way to create a JavaScript Object.
- Using an object literal, you both define and create an object in one statement.
- An object literal is a list of name:value pairs (like age:50) inside curly braces {}.
- Spaces and line breaks are not important. An object definition can span multiple lines.
- Example creates an empty JavaScript object, and then adds properties.

Example:

```
<p id="demo"></p>
<script>
const person = {firstName:"John", lastName:"Doe ", age:50,eyeColor:"blue"};
document.getElementById("demo").innerHTML =
person.firstName + " is " + person.age + " years old.";
</script>
```

```
<p id="demo1"></p>
<script>
const person1 = {};
person1.firstName = "John";
person1.lastName = "Doe";
person1.age = 50;
person1.eyeColor = "blue";
document.getElementById("demo1").innerHTML =
person1.firstName + " is " + person.age + " years old.";
</script>
```

➤ By using an object constructor

- It is considered good practice to name constructor functions with an upper-case first letter.
- In a constructor function this does not have a value. It is substitute for the new object. The value of this will become the new object, when a new object is created.

Example:

```
<p id="demo"></p>
<script>
// Constructor function for Person objects
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}
// Create a Person object
```

Unit-4 JavaScript Objects

```
const myFather = new Person("John", "Doe", 50, "blue");
```

```
// Display age
```

```
document.getElementById("demo").innerHTML =
```

```
"My father is " + myFather.age + ".";
```

```
</script>
```

- In the example above, function Person() is an object constructor function.
- Objects of the same type are created by calling the constructor function with the new keyword

Example:

```
<p id="demo"></p>
```

```
<script>
```

```
// Constructor function for Person objects
```

```
function Person(first, last, age, eye) {
```

```
    this.firstName = first;
```

```
    this.lastName = last;
```

```
    this.age = age;
```

```
    this.eyeColor = eye;
```

```
}
```

```
// Create two Person objects
```

```
const myFather = new Person("John", "Doe", 50, "blue");
```

```
const myMother = new Person("Sally", "Rally", 48, "green");
```

```
// Display age
```

```
document.getElementById("demo").innerHTML =
```

```
"My father is " + myFather.age + ". My mother is " + myMother.age + ".";
```

```
</script>
```

4.2 Date Object:

4.2.1 Date constructor:

Date()

- When a Date object is created, a number of methods allow you to operate on it.
- Date methods allow you to get and set the year, month, day, hour, minute, second, and millisecond of date objects, using either local time or UTC (universal, or GMT) time. (Greenwich Mean Time, Universal Time Coordinated)
- When you display a date object in HTML, it is automatically converted to a string, with the toString() method.

Example:

```
<p id="demo"></p>
```

```
<script>
```

```
const d = new Date();
```

```
document.getElementById("demo").innerHTML = d.toString();
```

```
</script>
```

Unit-4 JavaScript Objects

```
<p id="demo1"></p>
<script>
const d1 = new Date();
document.getElementById("demo1").innerHTML = d1.toUTCString();
</script>
```

```
<p id="demo2"></p>
<script>
const d2 = new Date();
document.getElementById("demo2").innerHTML = d2.toDateString();
</script>
```

```
<p id="demo3"></p>
<script>
const d3 = new Date();
document.getElementById("demo3").innerHTML = d3.toISOString();
</script>
```

Date(milliseconds)

JavaScript stores dates as number of milliseconds since January 01, 1970, 00:00:00 UTC (Universal Time Coordinated).

Example:

```
<p>100000000000 milliseconds from Jan 1, 1970, is approximately Mar 3, 1973:</p>
<p id="demo"></p>
<script>
const d = new Date(100000000000);
document.getElementById("demo").innerHTML = d;
</script>
```

Date(year, month, day, hours, minutes, seconds, milliseconds)

Example:

```
<p id="demo"></p>
<script>
const d = new Date();
document.getElementById("demo").innerHTML = d.getFullYear();
</script>

<p id="demo1"></p>
<script>
```

Unit-4 JavaScript Objects

```
const d1 = new Date();
document.getElementById("demo1").innerHTML = d1.getMonth() + 1;
</script>
```

```
<p id="demo2"></p>
<script>
const d2 = new Date();
document.getElementById("demo2").innerHTML = d2.getDate();
</script>
```

```
<p id="demo3"></p>
<script>
const d3 = new Date();
document.getElementById("demo3").innerHTML = d3.getHours();
</script>
```

```
<p id="demo4"></p>
<script>
const d4 = new Date();
document.getElementById("demo4").innerHTML = d4.getMinutes();
</script>
```

4.2.2 Date Methods:

Method	Description
getFullYear()	Get the year as a four digit number (yyyy)
getMonth()	Get the month as a number (0-11)
getDate()	Get the day as a number (1-31)
getHours()	Get the hour (0-23)
getMinutes()	Get the minute (0-59)
getSeconds()	Get the second (0-59)
getMilliseconds()	Get the millisecond (0-999)
getTime()	Get the time (milliseconds since January 1, 1970)
getDay()	Get the weekday as a number (0-6)

getDate()

Example:

```
<p id="demo"></p>
<script>
const d = new Date();
document.getElementById("demo").innerHTML = d.getDate();
</script>
```

Unit-4 JavaScript Objects

getDay()

Example:

```
<p id="demo"></p>
<script>
const d = new Date();
document.getElementById("demo").innerHTML = d.getDay();
</script>
```

getMonth()

Example:

```
<p id="demo"></p>
<script>
const d = new Date();
document.getElementById("demo").innerHTML = d.getMonth() + 1;
</script>
```

getHours()

Example:

```
<p id="demo"></p>
<script>
const d = new Date();
document.getElementById("demo").innerHTML = d.getHours();
</script>
```

setDate()

Example:

```
<p id="demo"></p>
<script>
const d = new Date();
d.setDate(15);
document.getElementById("demo").innerHTML = d;
</script>
```

setMonth()

Example:

```
<p id="demo"></p>
<script>
const d = new Date();
d.setMonth(4);
```

Unit-4 JavaScript Objects

```
document.getElementById("demo").innerHTML = d;  
</script>
```

toString()

Example:

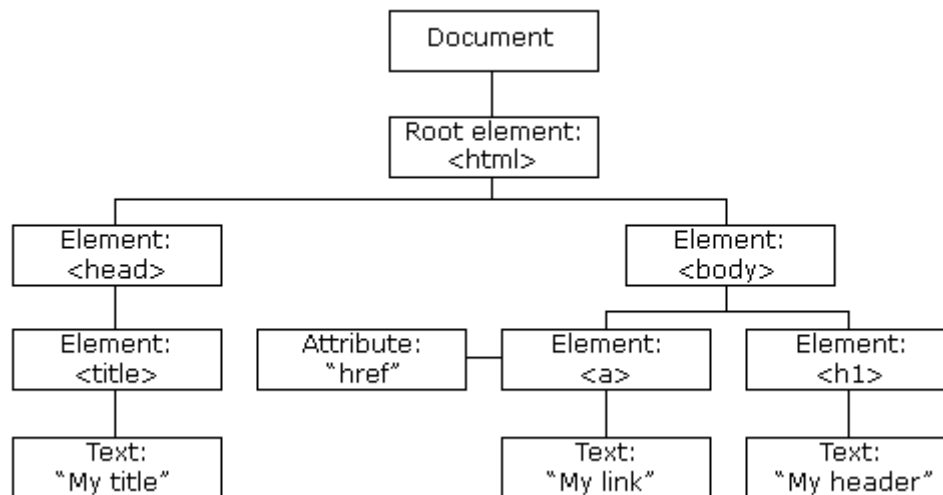
```
<p id="demo"></p>  
<script>  
const d = new Date();  
let text = d.toString();  
document.getElementById("demo").innerHTML = text;  
</script>
```

4.3 Document Object Model (DOM)

4.3.1 DOM concepts

With the HTML DOM, JavaScript can access and change all the elements of an HTML document.

When a web page is loaded, the browser creates a **Document Object Model** of the page. The **HTML DOM** model is constructed as a tree of **Objects**:



With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes
- JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

4.3.2 DOM properties:

Unit-4 JavaScript Objects

These are some typical DOM properties:

- x.innerHTML - the inner text value of x (a HTML element)
- x.nodeName - the name of x
- x.nodeValue - the value of x
- x.parentNode - the parent node of x
- x.childNodes - the child nodes of x
- x.attributes - the attributes nodes of x

Note: In the list above, x is a node object (HTML element).

4.3.3 DOM methods:

➤ write()

The write() method writes directly to an open (HTML) document stream.

Example:

```
<p>Write some HTML elements directly to the HTML output:</p>
<script>
document.write("<h2>Hello World!</h2><p>Have a nice day!</p>");
</script>
```

```
<h2>The Lorem Ipsum Passage</h2>
```

```
<p>
```

All text is delete when you clicked the button

```
<button type="button" onclick="myFunction()">Click me!</button>
```

```
<script>
```

```
function myFunction() {
    document.write("Hello World");
}
```

```
</script>
```

➤ writeln()

The writeln() method writes directly to an open (HTML) document stream.

Example:

```
<p>write() does NOT add a new line (CR) after each statement.</p>
```

```
<p>writeln() DOES add a new line (CR) after each statement.</p>
```

```
<pre>
```

```
<script>
```

```
document.writeln("Hello World!");
document.writeln("Have a nice day!");
```

```
</script>
```

```
</pre>
```

➤ getElementById()

- The getElementById() method returns an element with a specified value.
- The getElementById() method returns null if the element does not exist.

Unit-4 JavaScript Objects

- The `getElementById()` method is one of the most common methods in the HTML DOM. It is used almost every time you want to read or edit an HTML element.

Example:

```
<h1 id="demo">The Document Object</h1>
<script>
const myElement = document.getElementById("demo");
myElement.style.color = "red";
</script>
```

```
<h1 id="demo1">DOM</h1>
<script>
document.getElementById("demo1").style.color = "red";
</script>
```

➤ `getElementByName()`

The `getElementsByName()` method returns a collection of elements with a specified name.

Example:

```
<p>First Name: <textarea name="fname" type="text" value="Michael"></textarea></p>
<p>First Name: <input name="fname" type="text" value="Michael"></input></p>
<p>First Name: <textarea name="fname1" type="text" value="Michael"></textarea></p>
<p>The tag name of the first element with the name "fname" is:</p>
```

```
<p id="demo"></p>
<script>
let elements = document.getElementsByName("fname");
document.getElementById("demo").innerHTML = elements[1].tagName;
</script>
```